

MINIPYTHON

LENGUAJE DE PROGRAMACIÓN
IMPERATIVO

LIBRERÍAS IMPLEMENTADAS

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_VARS 100
```

TIPOS DE TOKENS

```
typedef enum {  
    TOKEN_NUMBER,  
    TOKEN_IDENTIFIER,  
    TOKEN_PLUS,  
    TOKEN_MINUS,  
    TOKEN_MUL,  
    TOKEN_DIV,  
    TOKEN_EQUAL,  
    TOKEN_PRINT,  
    TOKEN_LPAREN,  
    TOKEN_RPAREN,  
    TOKEN_END  
} TokenType;
```

ESTRUCTURA DE LOS TOKENS

```
typedef struct {  
    TokenType type;  
    char value[50];  
} Token;  
  
Token tokens[MAX_TOKENS];  
int token_count = 0;  
int current = 0;
```



```
typedef struct {  
    char name[50];  
    int value;  
} Variable;  
Variable variables[MAX_VARS];  
int var_count = 0;
```

ESTRUCTURA DE LAS VARIABLES

AST(ÁRBOL DE SINTAXIS ABSTRACTA)

TIPOS DE NODO

```
typedef enum {  
    NODE_NUMBER,  
    NODE_VARIABLE,  
    NODE_BINARY,  
    NODE_ASSIGN,  
    NODE_PRINT  
} NodeType;
```

ESTRUCTURA DEL ÁRBOL

```
typedef struct AST {  
    NodeType type;  
  
    int value;  
    char name[50];  
    char op;  
  
    struct AST* left;  
    struct AST* right;  
} AST;
```


FUNCIONES DEL ÁRBOL

CREAR NODO NUMERO

```
AST* create_number(int value) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_NUMBER;  
    node->value = value;  
    node->left = NULL;  
    node->right = NULL;  
    return node;  
}
```

CREAR NODO VARIABLE

```
AST* create_variable(char* name) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_VARIABLE;  
    strcpy(node->name, name);  
    node->left = NULL;  
    node->right = NULL;  
    return node;  
}
```

CREAR NODO PRINT

```
AST* create_print(AST* expr) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_PRINT;  
    node->left = expr;  
    return node;  
}
```

CREAR NODO DE UNA OPERACIÓN

```
AST* create_binary(char op, AST* left, AST* right) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_BINARY;  
    node->op = op;  
    node->left = left;  
    node->right = right;  
    return node;  
}
```

CREAR NODO ASIGNACIÓN

```
AST* create_assign(char* name, AST* value) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_ASSIGN;  
    strcpy(node->name, name);  
    node->left = value;  
    return node;  
}
```

CREAR NODO DE UNA OPERACIÓN

```
AST* create_binary(char op, AST* left, AST* right) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_BINARY;  
    node->op = op;  
    node->left = left;  
    node->right = right;  
    return node;  
}
```

CREAR NODO VARIABLE

```
AST* create_assign(char* name, AST* value) {  
    AST* node = (AST*)malloc(sizeof(AST));  
    node->type = NODE_ASSIGN;  
    strcpy(node->name, name);  
    node->left = value;  
    return node;  
}
```


MUCHAS GRACIAS

MINIPYTHON